

THE SECURITY OF A VLSI IMPLEMENTATION OF THE NEW AES AGAINST POWER AND TIMING ATTACKS

A THESIS SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF DOCTOR OF PHILOSOPHY
IN THE FACULTY OF SCIENCE AND ENGINEERING

2003

By
An Yu
Department of Computer Science

ProQuest Number: 10948350

All rights reserved

INFORMATION TO ALL USERS

The quality of this reproduction is dependent upon the quality of the copy submitted.

In the unlikely event that the author did not send a complete manuscript and there are missing pages, these will be noted. Also, if material had to be removed, a note will indicate the deletion.



ProQuest 10948350

Published by ProQuest LLC (2018). Copyright of the Dissertation is held by the Author.

All rights reserved.

This work is protected against unauthorized copying under Title 17, United States Code
Microform Edition © ProQuest LLC.

ProQuest LLC.
789 East Eisenhower Parkway
P.O. Box 1346
Ann Arbor, MI 48106 – 1346

(EF827)

✕
Th 23198 ✓

PREVIEW

JOHN RYLANDS
UNIVERSITY
LIBRARY OF
MANCHESTER

Contents

Abstract	9
Declaration	10
Copyright	11
Acknowledgements	12
1 Introduction	13
1.1 Publications	17
2 Cryptography	19
2.1 Cryptography History	19
2.2 Modern Ciphers	29
2.3 Modes of Operation	33
2.4 Data Encryption Standard (DES)	35
2.5 Summary	37
3 The Advanced Encryption Standard	39
3.1 Rijndael-128	41
3.2 Key Expansion	44
3.3 Round Key Addition	45
3.4 Byte Substitution	45
3.5 Shift Row	46
3.6 Mix Column	46
3.7 Summary	48
4 Asynchronous Circuit Design	49
4.1 Handshake Protocols	51

4.2	Data Encoding	52
4.3	Classes of Asynchronous Circuits	54
4.4	Summary	56
5	Attacks on Cryptographic Devices	57
5.1	Invasive Attacks	58
5.2	Non-invasive Attacks	59
5.2.1	Power Analysis	59
5.2.2	Electromagnetic Analysis	64
5.2.3	Timing Analysis	65
5.3	Security Issues for Smartcards	67
5.4	Summary	68
6	Rijndael Cryptosystem Design	69
6.1	Design Tools	69
6.2	Design Flows	70
6.3	Countermeasures Against Power and Timing Attacks	74
6.4	The Synthesised Rijndael Core	76
6.4.1	Rijndael Core	76
6.5	The Hand-designed Components	85
6.5.1	ROM	85
6.5.2	ROM Controller	88
6.5.3	RAM	90
6.5.4	RAM Controller	91
6.6	Summary	94
7	Evaluation of Single-rail Implementation	95
7.1	Preparations	95
7.2	Timing Analysis	96
7.3	Power Analysis	97
7.4	Summary	107
8	The dual-rail Rijndael Cryptosystem	108
8.1	The Balanced Design	109
8.2	ROM Controller and ROM	112
8.3	RAM Controller and RAM	116
8.4	Summary	119

9 Evaluation of Dual-rail Implementation	120
9.1 Power Analysis	120
9.2 Timing Analysis	127
9.3 The Origin of data-dependent Timing and Current	129
9.4 Summary	136
10 Conclusion	137
10.1 Research Summary	138
Bibliography	143
Glossary	150
A Rijndael Circuit Description in Balsa	153
A.1 The top-level Implementation	153
A.2 The Key Addition Routine	155
A.3 The 3-in-1 Routine	155
A.4 The Mix Column Routine	157
A.5 The Modular Multiplier Routine	158
A.6 The Key Expansion Routine	158
B Single-rail Schematics	160
B.1 The top-level Schematic	160
B.2 The ROM Controller	161
B.3 The RAM Controller	164
C Dual-rail Schematics	165
C.1 The top-level Schematic	165
C.2 The ROM Controller	166
C.3 The RAM Controller	169

List of Tables

2.1	The Trithemius square table	21
3.1	Time and cost for DES key search	40
3.2	Indices for bytes and bits	43
3.3	The Rijndael S-box	46
4.1	The 4-phase dual-rail encoding	53
6.1	Rijndael size and performance comparison	71
7.1	The current of XOR gates using different input data	100
9.1	Results of a 2-input completion detection tree	131
9.2	Results of 2-input dual-rail XOR and OR gates	132

List of Figures

1.1	The dependency map of chapters	18
2.1	Symmetric encryption and decryption	30
2.2	Substitution and permutation ciphers	30
2.3	Asymmetric encryption and decryption	31
2.4	Electronic codebook	33
2.5	Cipher-block chaining	34
2.6	Cipher feedback, r -bit characters/ r -bit feedback	34
2.7	Output feedback, r -bit characters/ n -bit feedback	35
2.8	Data encryption standard	36
2.9	The round transformation of DES	37
3.1	Mapping 16 bytes into 4×4 two-dimensional array	43
3.2	Rijndael-128 key expansion	45
3.3	The round key addition	45
3.4	The byte substitution transformation	46
3.5	The shift row transformation	46
3.6	The mix column transformation	47
4.1	A push channel handshake diagram	51
4.2	The handshake protocols	52
4.3	Push channel dual-rail handshake protocol	54
4.4	A 2-bit completion detection tree	54
4.5	A circuit with gate and wire delays	55
5.1	A CMOS inverter	60
5.2	Current trace of a Rijndael encryption	60
5.3	Different data vs current consumption	62
5.4	Differential power analysis on DES	64

6.1	The design flow for Rijndael-128	72
6.2	The architecture of the Rijndael cryptosystem	76
6.3	The 1-bit ROM	86
6.4	The single-rail ROM	87
6.5	The interconnect of the ROM and ROM controller	88
6.6	The ROM controller	89
6.7	The 32×16 RAM	90
6.8	The interconnect of the RAM and RAM controller	92
6.9	The RAM controller	92
6.10	The RAM read and write phases	93
7.1	Two current samples, A & B, and their difference, C	98
7.2	XOR current vs the sum of the hamming weights of inputs	101
7.3	Sum of hamming weights	102
7.4	ROM current consumption vs hamming weight of output	104
7.5	The key search	105
7.6	Current vs hamming weight of $state2_j$ for different assumed key_j	106
8.1	2-input dual-rail XOR and OR gates	111
8.2	The interconnect of the ROM and ROM controller	112
8.3	The dual-rail column decoder	114
8.4	The dual-rail row decoder	114
8.5	The 1-bit dual-rail ROM	115
8.6	The design of the Rijndael dual-rail ROM	116
8.7	The interconnect of the RAM and RAM controller	117
8.8	The dual-rail RAM controller	118
9.1	Current traces before synchronisation	121
9.2	Current traces after synchronisation	123
9.3	Dual-rail ROM current	124
9.4	The ROM current of the dual-rail Rijndael circuit	125
9.5	Current consumption comparison	127
9.6	The execution time and current of a 2-input CDT	130
9.7	The execution time and current of a 2-input dual-rail OR gate	133
9.8	The current of ROM accesses in schematic simulation	134
9.9	The schematic of an OR gate	135
9.10	The timing of a 2-bit completion tree using modified OR gates	136

B.1	The single-rail Rijndael	160
B.2	The ROM decoder	161
B.3	The 6to64 decoder	161
B.4	The 3to8 decoder	162
B.5	The 8to8 decoder	162
B.6	The 32to8 multiplexer	163
B.7	The 4to1 multiplexer	163
B.8	The RAM controller	164
C.1	The dual-rail Rijndael	165
C.2	The dual-rail ROM controller	166
C.3	The 6to64 dual-rail decoder	166
C.4	The 3to8 dual-rail decoder	167
C.5	The 8to8 decoder	167
C.6	The 32to8 dual-rail multiplexer	168
C.7	The 4to1 dual-rail multiplexer	168
C.8	The dual-rail RAM controller	169

Abstract

In recent years, new cryptanalytical techniques have been invented namely power and timing analysis, which pose a serious threat to cryptographic devices such as smartcards. By analysing the power dissipation or timing of encryptions in these devices, secret information inside can be deduced. The weakness, however, is not in the encryption algorithms themselves, but in their implementations. Almost every hardware devices incorporating encryption algorithms are vulnerable to power and timing attacks. These attacks take significantly less time than a brute-force search and they leave no trace that the device has been tampered with.

Power and timing attacks are real threats to cryptographic hardware, which cannot be simply ignored. AES, as the new encryption standard, will soon be adopted by many organisations such as banks and governments. Thus it is important to assess vulnerability of hardware implementations of AES and to introduce new designs to foil such attacks. This research implemented the secure AES cipher in asynchronous circuits. An analysis shows that an AES implementation in a single-rail design style is vulnerable to a power attack. The weakness is corrected by a dual-rail implementation. Experimental results show that this implementation has a considerable improvement in resistance against power attacks.

Declaration

No portion of the work referred to in this thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institution of learning.

PREVIEW

Copyright

Copyright in text of this thesis rests with the Author. Copies (by any process) either in full, or of extracts, may be made **only** in accordance with instructions given by the Author and lodged in the John Rylands University Library of Manchester. Details may be obtained from the Librarian. This page must form part of any such copies made. Further copies (by any process) of copies made in accordance with such instructions may not be made without the permission (in writing) of the Author.

The ownership of any intellectual property rights which may be described in this thesis is vested in the University of Manchester, subject to any prior agreement to the contrary, and may not be made available for use by third parties without the written permission of the University, which will prescribe the terms and conditions of any such agreement.

Further information on the conditions under which disclosures and exploitation may take place is available from the head of Department of Computer Science.

Acknowledgements

I would like to thank members of the AMULET group, especially Dr. Andrew Bardsley's support on various technical issues, comments on the first draft of this thesis and the help for designing a ROM.

I am also very grateful to the reviewers, my supervisor, Prof. David Brée (proof-reading), Dr. Steve Temple (proof-reading) and my adviser, Dr. Ning Zhang, who provided their valuable comments on this thesis. Also my thanks go to Dr. Luis Plana for sharing his view on the asynchronous circuits in terms of timing issues.

This research is sponsored by the Overseas Research Students Awards Scheme (ORS) and the Department of Computer Science. The support is greatly appreciated.

Finally, I would like to thank my wife and parents for years of continued support.

Chapter 1

Introduction

Cryptography is about code designing and code breaking. To keep secret information known only to the intended recipients, ciphers are designed which scramble the information into an unreadable form that can only be decrypted with the right key. As a secret is only shared within a small number of people, those who want to know the secret but don't have the key, try to break the code. This is how the game between code-designers and code-breakers started, and it has led to advances in cryptology.

Ciphers have undergone many changes. The primitive ciphers, invented two thousand years ago, encode messages by simply substituting for each letter another in the ciphertext or by shuffling the letters to different positions. This is not a true encryption as these ciphers don't have keys; they are easy to break by letter frequency analysis. As a result, a more advanced type of cipher, the polyalphabetic cipher, was invented, which allowed one plaintext letter to be substituted by different ciphertext letters depending on the context. This cipher doesn't have keys either, and so the same plaintext always resulted in an identical ciphertext. Hence, the design of the cipher had to be kept secret. This problem was finally solved by the invention of keys in 1500s. Now two parties could share secrets using a common cipher key and they could also change keys as desired. The concepts of polyalphabeticity and the key are still at the heart of the design of modern ciphers.

The security strength of a modern cipher is usually referenced to the length of its cipher key. For instance, a cipher supporting 128-bit keys means that there are 2^{128} possible keys to choose from. The combinations are enormous; if one were to try to decrypt a message by searching for the key that had been used, it

would take millions and millions of years even using today's computers. However the assumption here is that there are no shortcuts which could find a key using less effort than trying every possible key.

Thanks to advanced cryptanalysis techniques, such as differential and linear cryptanalysis, today we have a number of ciphers (e.g. 3DES, Rijndael, Serpent and Twofish, etc) that are regarded as strong ciphers. They have undergone years of extensive cryptanalysis by cipher experts and the cryptography community. These ciphers have no known hidden weakness or shortcuts and are considered to be computationally infeasible to break. In other words, the most efficient way to find an unknown key is a brute-force attack: searching every possible key.

With these trustworthy ciphers, are we safe from security breaches? The answer is 'no'. Although a cipher itself can be secure, a security product that uses such a cipher doesn't inherit the strength automatically. For instance, a house front door installed with the world's most secure lock won't necessarily prevent a break in, as there may well be other weakness in the house that an intruder can exploit, e.g. the windows. The house will only be as secure as the lock if there are no other weaknesses to be found.

In computer security, many security breaches fall into this type of example. For instance, some Unix systems use a modified DES cipher for user access authentication. The maximum length of a password is 8 characters, equivalent to 64 bits. The effective bits are only 56, as every eighth bit is ignored by the DES algorithm, that is 2^{56} possible combinations to choose from. However, certain passwords are weak (e.g. any word that can be found in a dictionary is considered weak, or the length of a password is short); some of them can be cracked within seconds by a dictionary attack. Even where strong passwords are used, security breaches can still happen. For example, a user runs the telnet program to access a remote computer, the password is transmitted in cleartext through the network as he logs in, so it can be captured by an adversary in the middle of the communication. There are many more such examples.

Similarly, cryptographic devices in hardware have similar problems. The reason for implementing cryptographic algorithms in hardware is that such devices are fast, reliable and difficult to duplicate. But more importantly, because of the difficulty in reverse engineering, the secret can be well hidden and protected. However no hardware is tamper-proof. Many techniques have been developed

that can safely depackage a silicon chip. The layout of a chip can then be reconstructed, using tools such as optical microscopes to produce high resolution photographs of the chip surface, and data, address bus lines, ROM, RAM, EEPROM, ALU, etc. identified. EEPROM (or recently Flash memory), where the secret is usually stored, can be read out using microprobing to observe the entire bus and to record the values in memory as they are accessed [35]; very often the built in sense amplifiers of the memories have to be used. Consequently, the secret information (e.g. master keys) is revealed. Depackaging and microprobing are not the only method that can compromise a chip's security. It inevitably leaves tamper evidence behind and it requires substantial skill and equipment in order to perform an attack. What's worrying are the techniques invented a few years ago, namely power and timing analysis. These cryptanalytic techniques study the behaviour of cryptographic devices through their execution time, power dissipation or electromagnetic emission, and then deduce the secret information. These types of attacks are difficult to detect because they don't involve tampering. Smartcards are particularly at risk, because they cannot be run in a monitored environment. A legitimate smartcard user can be an attacker, who can safely perform the attacks without being noticed by the card issuer. An even more worrying concern is that an attack can sometimes be undertaken very quickly and even without the owner of the card realising that it has happened. For example, when you are paying for goods or services in a shop, your credit card is inserted into a smartcard reader for processing. While the smartcard reader might look perfectly normal, it could actually be fitted with measuring devices which secretly make a series of measurements of the power consumed by your card; this data can be used to determine the secret keys inside the smartcard. The power measurement is done instantly, adding virtually nothing to the processing time you expected the normal transaction would take.

A power attack is surprisingly easy to perform. The equipment needed usually involves a data acquisition board (costing a few hundred pounds) or a digital oscilloscope as well as a PC. To probe the secret key in a device, an attacker needs to collect samples of instantaneous power dissipation as the target device is executing cryptographic operations. A small resistor is inserted in series with the power or ground input and the current is computed by dividing the voltage difference across the resistor by its resistance. When enough power samples are obtained, the attacker can then apply the power analysis technique to relate the

current to the internal state of the cipher, and very often, secret information, usually the cipher keys can be deduced. The number of power samples needed in an attack depends on the implementation of the cipher, which can vary from thousands (e.g. DES) to even a few tens of samples (e.g. Twofish) [16]. The power attack is a practical technique, which uses far fewer attempts than a brute-force attack. A timing attack is similar; it measures the time that a particular operation takes in a cipher, and relates the timing to the data that was processed. Consequently, cipher keys can be computed.

Power and timing attacks are real threats to cryptographic hardware, which cannot be simply ignored. In response to this problem, new designs are required to foil these attacks. This research aims to provide such a design. The cipher that was chosen was Rijndael, the Advanced Encryption Standard (AES). AES is a new encryption standard set by the US government to replace the ageing standard, Data Encryption Standard (DES), due to its 56-key being too short to prevent a brute-force attack. AES, however, uses 128-bit keys, which means that it would take millions and millions years to search for an unknown key using the computer power today. AES is likely to remain secure for some time (until a much more powerful computer is invented, e.g. Quantum computers). It is expected that AES will soon be accepted by governments, banks and many other organisations. As this algorithm is still quite new, there hasn't yet been much research based on power and timing analysis of hardware implementations of AES [10, 19, 16, 41]. This research is to build an AES cryptosystem in hardware, so that its vulnerability can be assessed and countermeasures can be put into place.

The AES cryptosystem I will be building is in asynchronous circuits, which is different from the usual, synchronous, circuit design style, where a global clock governs the entire circuit operation. The aim of this research is to verify whether or not an asynchronous AES implementation can achieve better security from the point of view of power and timing attacks. The asynchronous design style is chosen, because of a number of security advantages:

- Synchronous circuits are vulnerable to clock glitch attacks [4, 35]. Asynchronous circuit are clock-less, they are much less sensitive to this type of attack [44].
- Asynchronous circuits could reduce power consumption and electromagnetic emission, and so data-dependent information is likely to be reduced. [26, 7, 39, 50].

- Asynchronous dual-rail design style could reduce data-dependent power consumption by balancing the hamming weight of data. In the same way that data-dependent electromagnetic information could also be reduced [50].
- An extra benefit of the asynchronous dual-rail design style is that the circuit could be less sensitive to fault induction attacks [44, 65]. This benefit comes virtually free.

The thesis is in two parts: the first part introduces the necessary background information, and the second part presents the details of the experiments that have been done in the research. Chapter 1 (being this chapter) begins with a brief introduction of the security problem in the application of cryptography. In particular, hardware that implements encryption algorithms is vulnerable from implementation attacks even though the algorithm itself is computationally secure. Chapter 2 reviews the evolution of ancient cryptography and introduces modern cryptography, including the design of block ciphers and their modes of operation. The de-facto encryption standard DES, a block cipher is also examined in detail. Chapter 3 illustrates the AES algorithm that was implemented in hardware. Chapter 4 introduces two design styles of asynchronous circuits, which were used in my implementations. Chapter 5 describes the common threats to cryptographic hardware and how attacks can be performed.

Chapter 6 presents a single-rail AES implementation, from the design of encryption core to the ROM and RAM. Chapter 7 shows the results of the design from the point of view of power and timing attacks. Chapter 8 presents a dual-rail AES circuit, which addressed the security weakness found in the single-rail implementation. Chapter 9, presents the evaluation of the dual-rail version. Chapter 10 is the conclusion: hardware devices incorporating the AES need to be implemented in dual-rail.

1.1 Publications

The paper below has been submitted for publication:

- A Clock-less Implementation of the Advanced Encryption Standard Resistant to Power and Timing Attacks (Asiacrypt 2003)

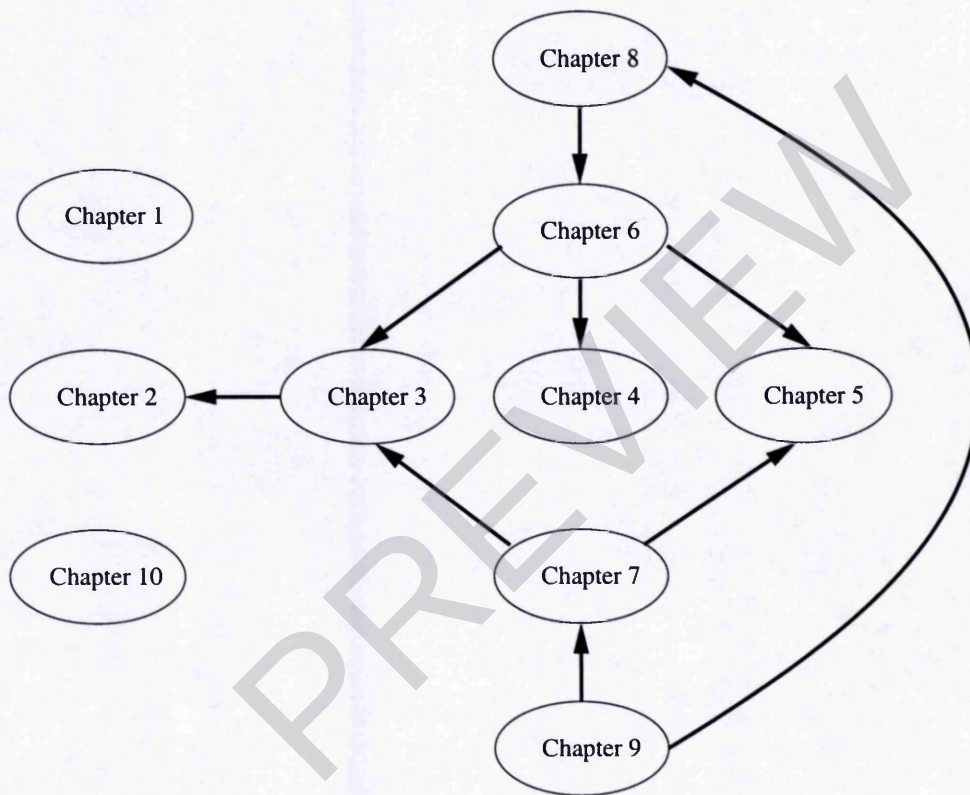


Figure 1.1: The dependency map of chapters

Chapter 2

Cryptography

This chapter reviews ancient cryptography (Section 2.1) and introduces modern cryptography (Section 2.2). The common modes of operation of a block cipher are described (Section 2.3). The de-facto encryption standard DES, a block cipher, is examined in detail (Section 2.4).

2.1 Cryptography History

Cryptography is the science of applying mathematics to encrypt and decrypt data. Cryptography enables sensitive information to be stored or transmitted across insecure communication channels (e.g. the Internet), so that the information cannot be read by anyone except the intended recipient.

Cryptography has a fascinating history. The first documented written cryptography was found about 4,000 years ago [31]. An Egyptian scribe used non-standard hieroglyphs in an inscription, found in the main chamber of the tomb of the nobleman Khnumhotep II. His inscription was not a system of secret writing as the modern world knows it. It only uses some unusual hieroglyphic symbols here and there in place of the more ordinary ones. The intention was not to make it hard to read the text, rather, it was to impart a dignity and authority to it. The inscription incorporated one of the essential elements of cryptography: a deliberate transformation of the writing.

About 50-60BC, the Roman emperor, Julius Caesar [31] used a simple substitution with the normal alphabet in government communications. This is a *Monoalphabetic Substitution Cipher*, which is merely shifting the letters a fixed

amount. Below is an example showing the ciphertext letters resulting from shifting the plaintext letters three times.

plaintext	a	b	c	d	e	f	g	h	i
ciphertext	D	E	F	G	H	I	J	K	L

In 1412, Shihāb Qalqashandi, an Egyptian, completed an enormous 14-volume encyclopaedia, which included a section on cryptology. The section named “*Concerning the concealment of secret messages within letters*”. It consists of two parts, one deals with symbolic actions and allusions, the other with invisible inks and cryptology [31]. Qalqashandi summarised a number of cipher systems, such as shifting letters a fixed amount, writing words backwards, reversing alternate letters of the words of a message, using figures to substitute for letters, etc. It is the first time in cryptography that both transposition (shifting letters in a message to different positions) and substitution ciphers were introduced.

A monoalphabetic substitution cipher, however, is trivial to break because letters in a plaintext have a different frequency. Statistics shows that the letter ‘e’ appears the most in a message written in English, and then the letter ‘t’, ‘n’, and so on. The letter frequency remains unchanged in the corresponding ciphertext due to the one to one relation between a plaintext letter and a ciphertext letter. Thus, the most frequent letter occurring in a ciphertext may well be the plaintext letter ‘e’ (or ‘t’ if not). The weakness of monoalphabetic substitution ciphers was realised by Aeon Battista Alberti and led him to invent the first *polyalphabetic substitution cipher* in 1466 (this type of cipher was apparently not broken until the 1800’s). In polyalphabetic substitution, ciphers as the name suggests, a plaintext letter are replaced by different ciphertext letters. The cipher works like a Caesar cipher using variable shifts. For instance, if one is to encrypt a message, one may choose to encrypt the first three words in a message with a 3-letter Caesar shift, and the following three words with a 4-letter shift and so on. In this cipher, the relationship between a plaintext letter and the corresponding ciphertext letter is variable. This cipher significantly increased the difficulty of code breaking. The invention of the polyalphabetic substitution cipher was a great leap forward in cryptography, as a consequence Alberti is often referred to as the Father of Western Cryptology.

Cryptography took another step forward in 1518, when Johannes Trithemius wrote his first printed book on cryptology. The book contains his contributions to polyalphabeticity. For the first time in cryptology, the square table appeared,

which is the elemental form of polyalphabetic substitution. Trithemius' square table (its first and last few rows) can be seen in Table 2.1:

plaintext	a	b	c	d	e	f	g	h	i	k	l	m	n	o	p	q	r	s	t	u	x	y	z	w
1	A	B	C	D	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	W
2	B	C	D	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	W	A
3	C	D	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	W	A	B
4	D	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	W	A	B	C
5	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	W	A	B	C	D
6	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	W	A	B	C	D	E
23	Z	W	A	B	C	D	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y
24	W	A	B	C	D	E	F	G	H	I	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z

Table 2.1: The Trithemius square table

The columns are chosen by the plaintext letters and the rows by the sequential position of the letter in the plaintext.

The rule is that, the first letter is encrypted with the first alphabet in the first row, the second letter is encrypted with the second alphabet in the second row and so on. So a plaintext 'attack' is 'auxdgp' in ciphertext. By shifting alphabets for each letter, the letter patterns such as double letters are concealed. The advantage of this cipher over Alberti's is that a new alphabet is brought into play with each letter, whereas in Alberti's cipher, alphabets are only shifted after a few words, therefore, the patterns of repeated letters can still be found. Trithemius also introduced regular alphabet changes as part of the enciphering algorithm. The Trithemius square was a great influence in cryptology.

In 1553, Giovan Batista Belaso advanced the polyalphabetic cryptology another level. He introduced the concept of a key, which he called a *countersign* [31]. The key may consist of some words and the words may be repeated as many times as desired. Supposing our key is 'HELLO' and the plaintext is 'a secret message'. To encrypt this plaintext, we shall put them on paper in this manner:

plaintext	a	s	e	c	r	e	t	m	e	s	s	a	g	e
key	H	E	L	L	O	H	E	L	L	O	H	E	L	L
ciphertext	H	Y	P	N	R	M	Z	Y	P	G	A	A	R	P

The keyletter that is paired with a given plaintext letter indicates the alphabet of the square table (see Table 2.1) that is to be used to encipher that plaintext letter. Thus, the plaintext is encrypted to 'H YPNRMZ YPGAARP'. This system

permits great flexibility. A message is no longer encrypted using a fixed order of standard cipher alphabets, but the order depends on the key being chosen. His design is the standard polyalphabetic cipher operation misnamed *Vigenère* by most writers to this day.

A famous book *De Furtivis Literarum Notis* was published by Giovanni Battista Porta in 1563 [31]. The book covers ancient ciphers, modern ciphers, cryptanalysis and a list of linguistic peculiarities. He classified ciphers as transposition, substitution and symbol substitution (letters were substituted by strange symbols). He also suggested the use of synonyms and misspellings to confuse the cryptanalysts. He appeared to be the first person in cryptology to use the idea of substitution of two letters by a single symbol.

In 1585, a book about ciphers, *Traicte des Cometes*, written by Blaise de Vigenère, introduced the first authentic plaintext and ciphertext autokey system. The autokey system used a Trithemius-like square table, a single letter (the first letter of the key) is known to both encipherer and decipherer; with it the decipherer can decrypt the first ciphertext letter and so get a start on his work. With this, he gets the first plaintext letter, then he uses this as the key to decrypt the second ciphertext letter, uses that plaintext letter as the key to decrypt the third ciphertext letter, and so on. The cipher now universally called the *Vigenère*, employs only standard alphabets and a short repeating keyword. Below is an example:

plaintext	a	t	o	p	s	e	c	r	e	t
key	T	A	T	O	P	S	E	C	R	E
ciphertext	X	I	A	H	T	V	U	S	G	I

In 1863, a book *Die Geheimschriften und die Dechiffrier-kunst*, written by Friedrich W. Kasiski, gave the first general solution of a polyalphabetic cipher with a repeating keyphrase, thus marking the end of several hundred years of strength for the polyalphabetic ciphers. The impressive 95-page volume of cryptanalysis has made an incredible revolution in cryptology, but unfortunately it seems to have received little notice at that time.

A theory of cryptology, *The Index of Coincidence* [31], was created by William Frederick Friedman in 1915. He introduced a statistical technique, which could discover the coincidences in cryptograms and eventually solve them. This technique has had a great influence in the modern cryptanalysis, opening the door